AI Continuity Bridge

> *"Your AI can change. Your work doesn't."*

Table of Contents

AI Continuity Bridge — Full Technical & Product Plan

A local-first desktop app that lets a developer's in-progress AI coding session survive interruption — usage limits, crashes, provider outages, or a deliberate switch — by capturing real project state and handing the task to another agent.

1 Product Architecture

Four layers, from raw capture to delivered handoff:

Layer	Purpose	What it stores / does
1. Session Archive	Raw capture	Every PTY session's stdin/stdout, timestamps, exit codes
2. Structured Project Memory	Facts, not chat	Objective, decisions, failed attempts, constraints, changed files
3. Context Compression	Make state handoff-sized	LLM-summarized snapshot of Layer 2, sized for a fresh agent's context window
4. Handoff Engine	Translate + deliver	Formats Layer 3 into a provider-specific prompt and re-launches the next agent

2 Tech Stack

Application Shell

Tauri **Rust core** **React / TypeScript**

Chosen over Electron: smaller binary, native OS-level access without bundling a full Chromium + Node runtime, and a materially smaller attack surface — important given this app touches source code, git history, and potentially secrets.

System / OS Layer — Rust Only

Rust **git2** **portable-pty** **native fs watchers**

- **File watching** — OS-native watchers (fsevents / inotify / ReadDirectoryChangesW)
- **Git operations** — `git2` (Rust bindings to libgit2) for diffs, status, branch/commit state
- **Process / PTY control** — spawns and controls CLI agents as child processes with full stdin/stdout/stderr access

Local State — SQLite

Structured project memory (Layer 2) and session archive (Layer 1) both live in embedded SQLite — zero ops, fully local, no server dependency.

Compression / Summarization — Python + FastAPI

A local microservice, called over localhost, doing LLM-based state compression using LangChain. The one layer where an LLM call is the tool, not the OS layer — kept isolated from the Rust core.

Agent Communication — CLI Wrapping (PTY), Not Browser Automation

Claude Code, Codex CLI, and Gemini CLI are spawned as child processes inside a PTY controlled by the Rust core. The app injects the handoff context as the opening input, and passively observes all terminal output for interruption-detection and activity-tracking.

> **WHY NOT C FOR THE OS LAYER**
> Rust already provides C-level OS control with memory safety. Given this app has direct access to source code, credentials, and shell execution, the security cost of introducing C — buffer overflows, manual memory management, FFI complexity — outweighs any negligible performance gain. One systems language, not two.

3 Trade-off Summary

Decision	Chosen	Rejected	Why
Shell	**Tauri**	Electron	Smaller, safer; Electron acceptable if speed-to-prototype wins
Systems language	**Rust only**	Rust + C	C adds memory-safety risk with no capability gain here
Agent comms	**PTY-wrap CLI**	Raw API-only	Keeps each agent's native tool-use loop; API-only forces rebuilding it
Web chat support	**Clipboard / extension autofill**	Browser automation / DOM scraping	Automation breaks on UI redesigns — flagged 10/10 platform risk
State store	**SQLite**	Postgres / cloud DB	Local-first privacy pitch requires no data leaving the machine
Search	**Structured queries**	Vector DB	Premature at MVP scale; add only if memory volume demands it

4 UI Design — The Live Activity Trace

As the wrapped agent runs, every observed action becomes a node in a live step tree — visually similar to the collapsible tool-use blocks in Claude.ai's interface or Claude Code's own terminal output.

4.1 Step Tree Example

```
▾ Session: Refactor auth (Claude Code)
   🕮 Reading auth.ts
   🕮 Reading db/schema.sql
   ✏ Editing auth.ts        [3 lines changed]
   ▾ 🔧 Running: npm test
      ✖ 2 failing — auth.test.ts
   ✏ Editing auth.test.ts
   ▾ 🔧 Running: npm test
      ✔ All passing
```

Each line is derived from two correlated signals: parsed PTY output (what command ran, what file path appeared) and the independent filesystem watcher (confirming a file's mtime actually changed). Cross-referencing both avoids showing "wrote a file" when the agent merely printed a code block without saving.

4.2 Headroom — Collapsing Process, Maximizing Outcome

The core UX problem: a long agent session generates far more step-detail than a user wants visible at once, but the summary/outcome should always stay prominent.

> **HEADROOM PATTERN**
>
> **Default collapsed state** — only the current/most recent 2–3 steps are expanded; older steps auto-collapse into a single summarized line ("12 earlier steps — 6 files touched, 1 error resolved").
>
> **Reserved space for outcome** — the process trace occupies a shrinking, scrollable region at the top; the live "what's happening now" line and eventual handoff summary always keep guaranteed space at the bottom.
>
> **Auto-promotion on interruption** — the same step data collapses directly into the handoff card ("64% progress, 8 files changed, 3 errors remaining") — nothing is re-derived.
>
> **Expand-on-demand** — clicking any collapsed group re-expands its steps, useful for debugging without cluttering normal operation.

5 MVP Scope — What Ships First

1. Tauri desktop shell, single monitored project folder
2. Rust-based file watcher + git state extraction
3. PTY wrapper for one agent: Claude Code

4. SQLite-backed structured memory — no compression service yet; raw structured state is small enough to hand off directly at MVP scale
5. Live activity tree UI with headroom-collapsing behavior
6. Manual interruption trigger → handoff card → manual "Continue with Codex CLI" (second PTY adapter)

> **SUCCESS CRITERION**
> A real interrupted coding task, picked up by the second agent, with the developer not having to re-explain the project. Validate with 5–10 real developers before adding a third agent, compression service, or any team/enterprise features.

AI Continuity Bridge — Technical & Product Plan. Internal working document.